ISSN 1009-3044
Computer Knowledge and Technology 电脑知识与技术
Vol.20, No.22, August 2024

E-mail：xsjl@dnzs.net.cn
http://www.dnzs.net.cn
Tel：+86-551-65690963 65690964

Spring Security+JWT 实现微服务架构中的身份验证和授权

吕玉桂

（广东白云学院，广东 广州 510000）

摘要：身份验证和授权是保障Web应用程序安全性的关键因素。文章以基于微服务架构的电商应用为例，从设计思路到实现过程，介绍了基于Spring Security框架结合JWT实现分布式环境下的身份验证和授权技术。系统采用RBAC模型实现了基于角色的动态授权，借助Java的jjwt库按照JWT规则生成Token，解决了分布式环境下的Token过期和Token传递问题，保证了服务之间的正确调用和良好的用户体验。

关键词：Spring Security；JWT；RBAC；微服务；身份验证；授权

中图分类号：TP311　　**文献标识码：**A

文章编号：1009-3044(2024)22-0060-04

DOI:10.14004/j.cnki.ckt.2024.1219

开放科学（资源服务）标识码(OSID)：

0 引言

随着互联网技术的飞速发展，其在各个行业的广泛应用日益广泛。随着用户数量的激增和业务场景的日益复杂化，传统的单体架构已难以满足当前互联网技术的发展需求。为了提升系统的可维护性和扩展性，并降低维护成本，微服务架构应运而生。该架构将一个大型软件应用分解为一组小型服务，每个服务均运行在其独立的进程中，使用轻量级机制通信，并独立部署每个服务，提升系统的维护性和扩展性。

微服务架构同样面临挑战，服务之间依赖API进行通信，因此确保API安全、限制未授权用户的访问以及数据篡改等至关重要。Spring Security作为安全框架提供了身份验证、授权、加密、会话管理等功能。JWT用于在多个服务之间传输加密的用户信息，进行身份验证和标识。RBAC基于角色实现对资源的访问控制。三者融合在一起实现微服务架构的安全策略。

本文以电商系统为例，介绍在微服务架构中使用Spring Security+JWT结合RBAC实现身份验证和授权。

1 核心技术

1.1 Spring Security

Spring Security是一个功能强大且高度可定制的基于Java的认证和授权框架，旨在为应用程序提供全面的安全保护。它提供了一系列的API和功能，帮助开发人员在应用程序中实现各种安全特性，如用户认证、授权、密码加密等。通过提供保护机制来防止常见的安全漏洞，CSRF，XSS、点击劫持等[1]，从而提高应用程序的安全性。核心原理是通过Spring AOP和Servlet过滤器来保护应用程序。当HTTP请求到达应用程序时，通过一组过滤器链进行不同的安全检查，验证用户身份以及是否有权限访问所请求的资源[2]。支持基于角色的访问控制，允许开发者定制安全策略。Spring Security强大的功能和灵活性使其成为保护Spring应用程序的首选安全解决方案。

1.2 JWT

JWT(JSON Web Token)是一种基于JSON格式的轻量级安全令牌标准，用于在网络应用中传递身份信息[3]，它由头部、载荷和签名三部分组成。用户登录后，服务器会生成JWT并发送给客户端。客户端将令牌存储在本地LocalStorage，后续携带JWT向服务器发送请求，服务器通过验证JWT确认用户身份[4]。JWT具有一定的有效期，过期后须重新获取新的令牌。JWT的跨域性、安全性、扩展性适合在分布式架构中实现身份验证。

1.3 RBAC

RBAC(Role-Based Access Control)基于角色的访问控制，是一种广泛应用于系统和网络安全中的访问控制模型[5]。RBAC模型引入了角色这一中间层概念，通过角色与权限的关联来简化用户与权限的直接关系，从而更灵活高效地管理系统的访问控制。通过将权限授予角色，可以减少权限分配的复杂性和重复性。当用户角色发生变化时，只需要更改角色与权限的关联，而无须管理每个用户的权限。RBAC模型以其灵活性、简化性、细粒度控制等优点成为权限控制的主要模型。

收稿日期：2024-03-13
基金项目：2023年度广东省本科高校教学质量与教学改革工程建设项目：现代产业学院，项目名称：白云宏产业学院（项目编号：BYSJ-ZL202304）
作者简介：吕玉桂(1976—)，女，甘肃武威人，软件设计师，本科，主要研究方向为软件工程、人工智能及大数据。

中国知网 https://www.cnki.net

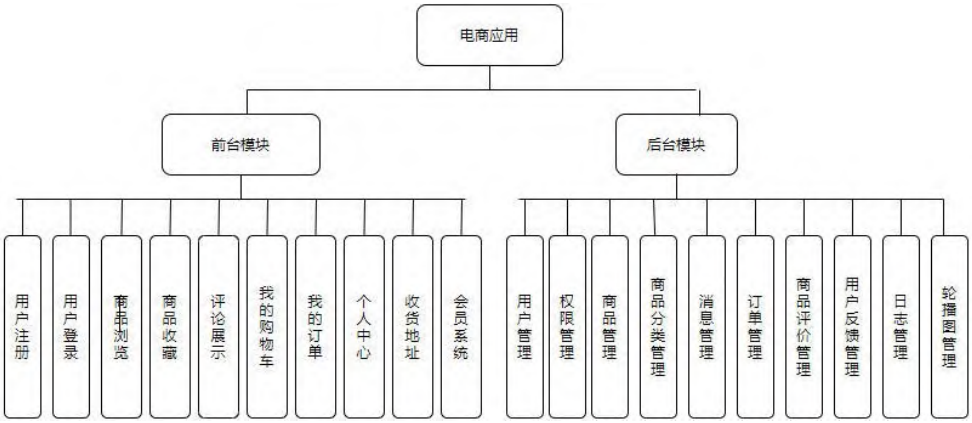

图1　系统功能模块图

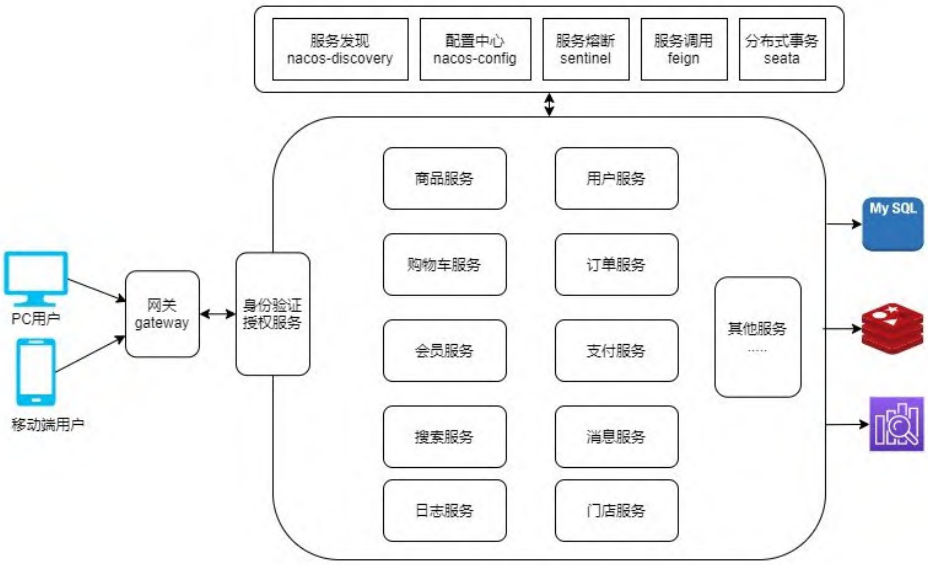

图2　系统架构图

2 系统架构

本文介绍的电商应用主要面向前台用户和后台的管理员,按照功能模块划分为前台模块和后台模块。具体功能模块如图1所示。

电商应用涉及功能多,需求变化快,业务复杂。传统的单体架构难以扩展,采用微服务架构根据功能拆分为独立的服务,单个部署,独立扩展,可以提高系统的稳定性和扩展性。微服务架构同样面临挑战,要解决服务间的通信、数据一致性、分布式事务处理等问题。本系统采取国内主流的微服务解决方案Spring Cloud+Spring Cloud Alibaba技术栈[3],从服务发现、配置、熔断、远程调用以及分布式事务全方位实现微服务应用。核心组件的应用有:Nacos:提供注册中心和配置中心,便于服务的发现和统一的配置管理。Sentinel: 解决服务熔断降级,流量控制,提升系统的可用性和稳定性。Feign: 简洁快速地实现微服务之间的通信和负载均衡。Seata: 以简单高效的方式解决分布式事务,让开发人员专注于业务逻辑。Gateway: 提供微服务统一的访问入口,实现权限拦截。服务列表:每个独立的功能模块,划分为一个独立服务。身份验证授权服务:验证用户身份并进行授权的服务。

系统架构如图2所示。

本系统面向的PC端用户和移动端用户拥有不同的权限,每当请求后端服务时,都需要身份信息。身份验证是用户访问后端服务的必经之路,为提升性能,会独立设计身份验证授权服务实现用户身份及权限的过滤。

3 身份验证以及授权实现

3.1 数据库设计

本系统面向管理员用户和终端小程序用户,两者需要存储不同的用户信息以及权限,因此独立设计会员表和系统用户表,根据RBAC设计基于用户和角色权限都是多对多关系。设计用户相关表如图3所示。

3.2 网关设计

网关作为用户访问后端服务的第一道关卡,主要用于实现用户拦截。用户在服务器登录成功时,根据JWT生成Token返回到客户端,并保存在LocalStorage[4]。用户向后端发请求时须携带Token,网关则验证用户是否在请求头中包含了Token,如果有,则放行,否则拦截。

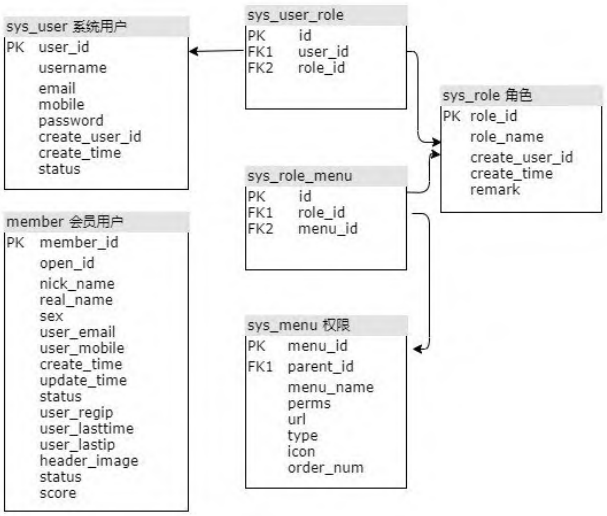

图3　用户权限数据表结构

中国知网 https://www.cnki.net

在网关服务中，创建TokenCheckFilter实现GlobalFilter接口，提供token的验证。用户在访问时，通过请求头header携带Authorization：Bearer Token。如果Token合法并且在Redis中存在，则继续进行。否则直接拦截在网关之外。代码思路如下：

```
public class TokenCheckFilter implements GlobalFilter, Ordered {
public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {    //1.判断是否是可以放行的url
//2.从header中获取身份授权
//3.获取token的实际内容--将前缀Bearer替换为""
//4.如果token合法，并且在redis中存在(没有过期)则放行
//5.如果没有合法的token,则拦截在网关之外
//6.输出结果 }
```

3.3 身份验证授权服务

授权服务用于用户身份信息及授权验证。如果用户第一次登录，则需要提供用户名密码或者小程序用户信息进行数据库验证。如果提供的信息正确，则根据JWT生成Token，存到redis，并将token信息返回前端，存入LocalStorage。其中，Spring Security框架负责登录验证，用户注销，加密处理以及相关安全的配置。

3.3.1 登录验证

采用Spring Security的验证功能，自定义实现接口UserDetailsService接口，提供登录验证。本项目有两种类型用户，使用同一套登录验证程序。因此在登录时使用userType=sysUser或者userType=Member分别进行身份验证和授权处理，即同一套验证登录程序处理两种类型的用户登录[5]。代码思路如下：

```
public class UserDetailServiceImpl implements UserDetailsService {
//小程序传递header信息loginType=Member,后台用户loginType=SysUser
public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
//获取用户类型
switch (loginType) {
case AuthConstant.SYS_USER: //后台系统用户
{//根据用户名查询表sys_user,sys_role,sys_menu,sys_user_role,sys_role_menu
//获取用户以及权限并返回系统用户对象
}
case AuthConstant.MEMBER:
{ //调用小程序接口获取open_id,查询数据库如果有直接返回
//如果不存在则自动注册插入数据库，返回会员对象 }
}
return //系统用户或者会员; } }
```

3.3.2 安全验证

继承使用Spring Security的WebSecurityConfigurer-Adapter类实现安全验证功能，处理登录，注销以及加密等逻辑。

```
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {
@Bean //登录成功的处理逻辑
public AuthenticationSuccessHandler authenticationSuccessHandler()
{ // 生成token，返回前端，同时保存在redis }
@Bean //处理登录失败的逻辑
public AuthenticationFailureHandler authenticationFailureHandler()
{ //设置错误信息，并将结果写入json对象，输出到客户端 }
@Bean //注销成功处理逻辑
public LogoutSuccessHandler logoutSuccessHandler()
{ //从header中获取token，获取真正的token，删除redis缓存 }
}
```

3.4 Token的解析和续约

用户访问后端服务时携带的Token仅包含用户信息，用户的权限信息存储在服务端Redis中。Spring Security进行身份验证时需要用户对象以及权限集合，需要将Redis中的用户级权限字符串反序列化为用户对象，转换成Security的身份对象，存入Security上下文，保证后续的操作可用，这个过程就是Token解析。同时，为了提升用户体验，在用户持续访问网站的过程中，Token过期时间自动后延，避免绝对过期，即Token的续约。使用过滤器技术，实现接口OncePerRequestFilter，在每次请求时进行Token解析和续约操作。代码思路如下：

```
public class TokenTranslateFilter extends OncePerRequestFilter {
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain) throws ServletException, IOException {
//1.获取路径，判断是否需要放行
//2.解析token
//3.续约操作，设置redisTemplate.expire(token,过期时间);
//4.将token转换成用户对象放入security，获取登录类型，转换为会员或者管理员
//5.将token字符串解析为身份令牌对象，登录成功时存入authentication对象
//6.将认证对象保存到security上下文
}}
```

3.5 远程服务Token传递

微服务架构中经常涉及服务之间的调用，在本案

中国知网 https://www.cnki.net

例中如订单服务会调用商品服务。用户在调用订单服务时提供了合法的token,但是订单服务使用OpenFeign调用商品服务时,用户在发送完请求后,Header中的Token已经丢失,因此订单服务内部调用商品服务时会导致远程调用失败。为了保证在远程调用时将原始服务携带的Token传递到目标服务,通过实现OpenFeign组件提供的RequestInterceptor拦截器接口修改请求头,将请求头header中的用户Token信息传递到目标服务中。代码思路如下:

```
public class FeignInterceptor implements RequestInterceptor {
    public void apply(RequestTemplate template) {
    //1.获取请求对象
    //2.从请求头中获取Token和用户类型
    //3.修改请求模版,将Token和用户类型传递到后续服务
    //4. template. header(AuthConstant. AUTHORIZATION, token)
            }
        }
    }
}
```

以上关于身份验证设计思路使用Spring Security解决了用户登录验证、加密、权限拦截的问题。JWT将用户信息加密生成Token标识用户身份,实现了微服务之间的用户身份传递,使用RBAC模型,动态授权,实现了基于角色的用户权限,三者结合解决了微服务架构的用户安全。

4 结束语

本文中电商项目自定义授权服务,采用Spring Security提供的身份验证和授权等解决方案保护了Web应用程序的安全性。结合JWT将用户信息通过签名生成Token令牌保存在客户端,同时将用户及权限信息序列化之后存入Redis,并处理了Token的校验解析以及续约,保证了微服务之间的正确调用。在测试中,本项目身份验证安全可靠且体验良好。项目涉及的业务逻辑有一定规模和复杂度,使用微服务架构解决了扩展性但同时项目架构及身份安全控制也变得更加复杂。未来,将会继续探索和研究关于分布式环境中应用程序安全的实现技术,同时持续优化关于微服务架构的安全实现,提升执行效率,封装为可重用的安全组件,快速地实现分布式架构的安全保证。

参考文献:

[1] 李立猛.Spring Security解决登录安全问题的方案[J].电子元器件与信息技术,2023,7(1):110-113.
[2] 周虎.一种基于JWT认证token刷新机制研究[J].软件工程,2019,22(12):18-20.
[3] 王悦,张雷,钱英军.基于SpringBoot微服务的Spring Security身份认证机制研究[J].电脑编程技巧与维护,2019(8):64-65,68.
[4] 肖双林,何迎生,田杰,等.基于JWT+Spring Security的动态权限管理系统[J].信息与电脑,2021(14):131-134.
[5] 王松.深入浅出Spring Security[M].北京:清华大学出版社,2021.

【通联编辑:代影】

(上接第59页)

3 结论

针对卷烟稽查过程中所面临的信息处理与管理问题,本文基于微信小程序开发平台设计并开发了实际现场能够方便快捷操作的卷烟稽查现场取证信息管理系统。稽查人员可以通过前端微信小程序对现场稽查情况进行实时的拍照上传;管理人员使用后端Web管理平台对前端微信小程序上传的稽查数据进行审核和导出,同时可视化组件能够清晰直观地对数据进行统计分析。卷烟稽查现场取证信息管理系统的主要功能模块在实际的卷烟稽查现场进行了使用测试,系统的交互性及稳定性良好,具备较好的可用性。

参考文献:

[1] 王小波.卷烟市场监管实效性实现的根本途径分析[J].法制与社会,2014(22):201-203.
[2] 邓艳.卷烟市场监管工作现状分析及完善措施探讨[J].企业改革与管理,2022(2):171-173.
[3] 朱志慧,蔡洁.基于SpringBoot+Vue+Uni-app框架的校园失物招领系统[J].电子技术与软件工程,2022(17):62-65.
[4] 章跃庭.Vue-Element-Admin在广播电视员工线上培训系统中的应用[J].电视技术,2020,44(12):1-3.
[5] 沈涛,胡旭辉.基于JSP平台的高校学生工作管理系统的实现[J].信息与电脑(理论版),2020,32(5):105-107.
[6] 邹明荣,刘小玲,黄琨,等.基于WXSS/WXML技术的景区微信小程序的开发——以西岭雪山为例[J].信息系统工程,2020(2):76-79.
[7] 艺龙网信息技术(北京)有限公司.微信小程序页面动态化方法、系统、设备及储存介质:中国,CN202311523881.0[P].2024-02-20.
[8] 黄文杰,殷思琴.基于MVVM模式的实验设备开放共享管理系统设计[J].长江信息通信,2023,36(6):117-119.
[9] 崔庆森,王烁宇,贾裕民,等.基于云开发的藏汉双语跑步微信小程序的设计与实现[J].自动化应用,2023,64(14):221-223.
[10] 岑俊慷,刘卫.基于微信小程序的在线答题系统的设计与实现[J].现代计算机,2023,29(10):81-86.
[11] 张敏,覃凯莉,陈彪,等.基于微信小程序的在线应急语言服务平台研究与设计[J].西南民族大学学报(自然科学版),2023,49(1):94-104.
[12] 钟亮,彭毅.卷烟物流设备健康管理系统设计与实现[J].物流科技,2021,44(12):58-60,65.

【通联编辑:谢媛媛】

中国知网 https://www.cnki.net